Course Code	**CS3623**
Course Title	**Advanced Computer Programming**
Credit Hours	**3(2+1)**
Prerequisites	**Object Oriented Programming**

Weekly Lab Plan:

Week	Topics	Activities

1	Introduction To IDE, Introduction to java and its component
	Brief history of Java, Features/characteristics of Java, Java compilation Process, Types of Java Applications, Java Development Kit, Java Editions, and Java Development Tools
		Lab Task:
		1. Install and configure Java Development Kit (JDK) on your system
		2. Install an IDE (Eclipse/IntelliJ/NetBeans) and explore its features
		3. Create, compile and run a simple "Hello World" program using:
		  o Command prompt
		  o IDE
		4. Experiment with basic Java syntax by modifying the Hello World program

2	Difference between JRE, JDK, JVM, and JIT, Java Execution Flow Compiling and Execution of Java Program through Command Prompt, path and CLASSPATH, Anatomy of Java Program	**Lab Task:** 1. Write a program that demonstrates different primitive data types 2. Create a program that uses various operators (arithmetic, relational, logical) 3. Develop a simple calculator program that takes two numbers and performs basic operations 4. Experiment with Scanner class for console input and JOptionPane for dialog input
		Lab Task:

3	To obtain input from the console using the Scanner class To obtain input using the JOptionPane input dialog boxes To use identifiers to name variables, constants, methods, and classes To use constants to store permanent data To declare Java primitive data types: byte, short, int, long, float, double, and char To use Java operators to write numeric expressions To use short hand operators To cast value of one type to another type To represent a string using the String type To demonstrate defining classes and creating objects To create objects using constructors To access objects via object reference variables To define a reference variable using a reference type To access an object's data and methods using the object member access operator (.) To define data fields of reference types and assign default values for an object's data fields	1. Create a Student class with attributes (name, rollNo, age) and methods 2. Implement a BankAccount class with deposit/withdraw methods 3. Create multiple objects of these classes and demonstrate their usage. 4. Write a program showing method overloading
4	Review of OO programming using java, Inheritance, Polymorphism Encapsulation, and Abstract and Interface classes	**Lab Task:** 1. Implement inheritance with a Person->Student->GraduateStudent hierarchy 2. Create an interface and implement it in a class 3. Demonstrate polymorphism through method overriding 4. Create an abstract class with concrete and abstract methods
	Abstract method and an Abstract class, Interface, Interface as a Type, Interface vs. Class, defining an Interface,	**Lab Task:**

5	Implementing an Interface, Implementing multiple Interface's, Inheritance among Interface's Overview of exceptions and exception handling, To explore the advantages of using exception handling, To write a try-catch block to handle exceptions, To use the finally clause in a try-catch block, Checked vs Unchecked Exceptions, To declare exceptions in a method header, To throw exceptions in a method, To re-throw exceptions in a catch block, To define custom exception classes	1. Write programs demonstrating try-catch-finally blocks 2. Create custom exception classes and throw them in a program 3. Handle different types of exceptions (checked and unchecked) 4. Implement a program that reads numbers from a file with proper exception handling
6	File and I/O Stream Object Serialization and Externalization	**Lab Task:** 1. Write a program to read from and write to text files 2. Create a program that serializes and deserializes objects 3. Implement a simple contact book that saves/loads contacts to/from a file 4. Compare different I/O stream classes
7	Graphical User Interface (GUI) using Swing, GUI Helper classes, Creation of GUI Components Frames, Layout Manager, Layout Manager Types, Flow Layout, Grid Layout, Border Layout, Multiple Panels in a Frame, JPanel and Graphics, Multiple Panels in a Frame, Color Class,	**Lab Task:** 1. Create a simple window with basic components (JButton, JLabel, JTextField) 2. Experiment with different layout managers (FlowLayout, BorderLayout, GridLayout) 3. Build a temperature converter GUI (Celsius to Fahrenheit) 4. Create a form with multiple input fields and a submit button

8	Procedural vs Event-Driven Programming, Event classes, Event Information, Selected User Actions, Delegation Model, Inner class listener, The delegation model with inner class listener, Anonymous inner classes, Event Handler. Using coding	**Lab Task:** 1. Implement action listeners for buttons in a calculator GUI 2. Create a program that changes background color based on user selection 3. Develop a simple drawing application with mouse events 4. Build a GUI with multiple event listeners (mouse, keyboard, button)
	MID TERM EXAM	
9	Building GUI Through Drag And Drop, Use Of Event Listeners Building Fully Functional GUIs With Event Listeners And Layout Managers, Java Wrapper Classes, Writing Programs Using String And Associated Classes And Important String Functions	**Lab Task:** 1. Create a student registration form with various Swing components 2. Implement a menu system with file operations 3. Build a simple text editor with file open/save functionality 4. Create a GUI that uses JTable to display tabular data
10	JDBC, Relational Databases, ODBC, SQL Statements Inserting, Updating	**Lab Task:** 1. Set up a database connection with MySQL/JavaDB 2. Create a program that executes simple SQL queries 3. Implement CRUD operations for a student database 4. Write a program that displays database metadata
	Deleting Records, Meta Data	**Lab Task:**

11	JDBC Revision	1. Implement prepared statements for secure database operations 2. Create a program that handles transactions 3. Build a GUI front-end for database operations 4. Implement a simple login system with database authentication
12	Using Lambda Expressions Example of Lambda Expression	**Lab Task:** 1. Convert anonymous inner classes to lambda expressions 2. Implement functional interfaces using lambdas 3. Use lambda expressions with Java's built-in functional interfaces 4. Create a program that uses lambda expressions for event handling
13	Concurrency and Threading, single thread Multi-threading	**Lab Task:** 1. Create and run simple threads by extending Thread class 2. Implement Runnable interface to create threads 3. Demonstrate thread lifecycle states 4. Create a program showing thread priorities
14	Network Programming TCP Network Programming UDP	**Lab Task:** 1. Implement thread synchronization for a shared resource 2. Create a producer-consumer problem solution

		3. Develop a simple TCP client-server chat application 4. Implement a UDP time server-client program
15	Distributed Applications using RMI Documenting and packaging java applications	**Lab Task:** 1. Create a simple RMI application 2. Implement a remote calculator service 3. Package a Java application as a JAR file 4. Generate JavaDoc documentation for a sample project
16	Project Presentation	**Lab Task:** • Finalize and demonstrate course projects • Peer review of projects • Project code walkthroughs Each lab session should include: 1. Demonstration of key concepts by the instructor 2. Hands-on implementation by students 3. Debugging exercises 4. Small challenges to reinforce learning 5. Review and Q&A session
	FINAL TERM EXAM	